A PROJECT REPORT

ON

CAMPUS PLACEMENT AND INTERNSHIP PORTAL

(An Enterprise Java Web Application for University Recruitment Drives)

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

SUBMITTED BY:

1. Adhiraj Singh Shekhawat (24E1ARITM40P003)
2. Shlok Verma (24E1ARITM40P057)
3. Saurabh Gupta (24E1ARITM40P054)
4. Krishna Dahima (24E1ARITM40P027)
5. Himanshu Yadav (Roll: [COLLEGE RECORD])

UNDER THE GUIDANCE OF:

Faculty Mentor / Project Guide

Department of Information Technology

Arya College of Engineering & IT

Jaipur, Rajasthan

Academic Session: 2026 – 2027

DEPARTMENT OF INFORMATION TECHNOLOGY

ARYA COLLEGE OF ENGINEERING AND IT

[Affiliated to Rajasthan Technical University, Kota]

Kukas, Jaipur, Rajasthan – 302028

ARYA COLLEGE OF ENGINEERING AND IT

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the Project-Based Learning (PBL) report entitled "CAMPUS PLACEMENT AND INTERNSHIP PORTAL" is a bona fide record of work carried out by Adhiraj Singh Shekhawat (24E1ARITM40P003), Shlok Verma (24E1ARITM40P057), Saurabh Gupta (24E1ARITM40P054), Krishna Dahima (24E1ARITM40P027), and Himanshu Yadav (Roll No: [Verify from College Records]), students of Bachelor of Technology in Information Technology, 5th Semester, at Arya College of Engineering and IT, Jaipur, affiliated to Rajasthan Technical University, Kota.

The project has been executed in partial fulfillment of the academic curriculum prescribed for the degree of Bachelor of Technology in Information Technology during the academic session 2026-2027. The project embodies the implementation of Core Java, JavaServer Pages (JSP), Servlets, JDBC architecture, and MySQL relational database, conforming to established academic and industry standards.

_____	_____	_____
Faculty Mentor **Dept. of IT**	**Project Coordinator** **Dept. of IT**	**Head of Department** **Dept. of IT**
Date: [DD/MM/YYYY] Place: Jaipur	Date: [DD/MM/YYYY] Place: Jaipur	Date: [DD/MM/YYYY] Place: Jaipur

CANDIDATE DECLARATION

We hereby declare that the work presented in this Project-Based Learning (PBL) report entitled "CAMPUS PLACEMENT AND INTERNSHIP PORTAL" submitted to the Department of Information Technology, Arya College of Engineering and IT, Jaipur, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, is an authentic record of our original technical work carried out under the supervision and guidance of our esteemed faculty mentors.

We further declare that the content, system architecture, database design, and software artifacts detailed in this document have not been submitted either in part or full to any other university or institute for the award of any other degree, diploma, or academic distinction. All external sources, code libraries, documentation, and reference materials have been duly acknowledged and cited in accordance with institutional guidelines.

Team Member Name	Roll Number	Signature
1. Adhiraj Singh Shekhawat	24E1ARITM40P003	_____
2. Shlok Verma	24E1ARITM40P057	_____
3. Saurabh Gupta	24E1ARITM40P054	_____
4. Krishna Dahima	24E1ARITM40P027	_____
5. Himanshu Yadav	[Verify College Records]	_____

Date: [DD/MM/YYYY]
Place: Arya College of Engineering and IT, Jaipur

ACKNOWLEDGEMENT

The completion of this Project-Based Learning project on "Campus Placement and Internship Portal" has been a rewarding and intellectually stimulating technical endeavor. We take this opportunity to express our deep gratitude and sincere appreciation to all individuals who provided guidance, technical assistance, and administrative encouragement throughout the conception, development, and deployment phases.

First and foremost, we express our profound gratitude to our project mentor, whose constant encouragement, critical reviews, and incisive feedback helped us overcome architectural challenges in Java Servlets, JDBC connection management, and MySQL transaction isolation. Their guidance ensured our application conformed strictly to the principles of robust Model-View-Controller (MVC) software engineering.

We extend our special thanks to the Head of Department (HOD) and the Project Coordinator, Department of Information Technology, for establishing high academic standards and providing state-of-the-art laboratory infrastructure and computing facilities. We are equally grateful to the leadership and management of Arya College of Engineering and IT for fostering an environment conducive to innovation and practical technical education.

Lastly, we express our heartfelt appreciation to our peer team members, lab assistants, and family members for their patience, mutual technical support, and constructive suggestions during long debugging and testing sessions.

PROJECT TEAM MEMBERS:

Adhiraj Singh Shekhawat | *Shlok Verma* | *Saurabh Gupta* | *Krishna Dahima* | *Himanshu Yadav*

ABSTRACT

University campus recruitment and internship drives constitute a mission-critical operational vertical in modern technical institutions. However, conventional campus placement processes rely heavily on manual spreadsheets, unstructured email communications, and disjointed Google Forms. These archaic practices suffer from severe drawbacks, including high rates of data redundancy, human error in eligibility verification, missed recruitment deadlines, and a critical absence of real-time application tracking for aspiring student candidates.

To resolve these operational bottlenecks, this project presents the "Campus Placement and Internship Portal" (Campus Career Hub)—an enterprise-grade, full-stack web application developed using pure Core Java, JavaServer Pages (JSP), Java Servlets, JDBC, and MySQL. Adhering strictly to the three-tier Model-View-Controller (MVC) design pattern, the system empowers three core stakeholders: Students, Corporate Recruiters, and the Training and Placement Officer (TPO/Admin).

Key technical innovations include an Automated Server-Side Eligibility Validation Engine that programmatically checks student academic thresholds (CGPA, backlogs, branch) before permitting job submissions, cryptographic password protection using adaptive jBCrypt hashing, ACID-compliant database transaction management with automatic rollback mechanisms, and a responsive, high-contrast dark-mode SaaS user interface. The system has been containerized using a multi-stage Docker build and successfully deployed on the Render cloud platform with remote managed MySQL storage.

Keywords:
Campus Recruitment, Java Servlets, JSP, JDBC, MVC Architecture, MySQL, Role-Based Access Control, Docker Containerization, Cloud Deployment, Render.

TABLE OF CONTENTS

Chapter No.	Chapter Title / Topic Description	Page No.
Preliminary	Title Page, Certificate, Declaration, Acknowledgement, Abstract	1 – 5
Preliminary	Table of Contents & List of Figures / Tables / Abbreviations	6 – 7
Chapter 1	INTRODUCTION (Background, Need, Problem Statement, Objectives, Scope)	8 – 10
Chapter 2	REQUIREMENT ANALYSIS (Stakeholders, Functional, Non-Functional, Feasibility)	11 – 13
Chapter 3	SYSTEM ANALYSIS AND DESIGN (Architecture, DFD 0/1, UML Diagrams, ERD)	14 – 19
Chapter 4	TECHNOLOGY STACK (Java, JSP, Servlets, JDBC, MySQL, Tomcat, Maven, UI)	20 – 22
Chapter 5	MODULE IMPLEMENTATION (Auth, Student, Recruiter, Admin, DAO Layer)	23 – 29
Chapter 6	DATABASE DESIGN & IMPLEMENTATION (Schema, Integrity, SQL Queries)	30 – 32
Chapter 7	SECURITY AND VALIDATION (BCrypt Hashing, SQLi Prevention, AuthFilter)	33 – 34
Chapter 8	SOFTWARE TESTING (Strategy, Unit, Integration, Functional Test Cases)	35 – 38
Chapter 9	USER INTERFACE AND RESULTS (SaaS Dark UI, Live Screenshots & Results)	39 – 42
Chapter 10	PROJECT MANAGEMENT (WBS, Team Contributions, 3-Month Plan, Git Flow)	43 – 45
Chapter 11	DEPLOYMENT & MAINTENANCE (Docker Container, Render Cloud Hosting)	46 – 47
Chapter 12	CONCLUSION, FUTURE SCOPE, REFERENCES & APPENDICES	48 – 50

LIST OF FIGURES, TABLES AND ABBREVIATIONS

LIST OF FIGURES

Figure 3.1: 3-Tier MVC System Architecture Diagram ..... Page 14
Figure 3.4: Data Flow Diagram - Level 0 (Context Level) ..... Page 15
Figure 3.5: Data Flow Diagram - Level 1 (Functional Decomposition) ..... Page 15
Figure 3.6: System Use Case Diagram (Actors & Interactions) ..... Page 16
Figure 3.7: Student Job Application Activity Diagram ..... Page 16
Figure 3.8: Sequence Diagram - Job Application & Eligibility Check ..... Page 17
Figure 3.9: Software Component Diagram (MVC Layers) ..... Page 17
Figure 3.10: Entity Relationship Diagram (ERD) with Multiplicities ..... Page 18
Figure 3.12: Relational Database Foreign Key Relationship Map ..... Page 19
Figure 5.22: Inter-Module Integration Architecture Diagram ..... Page 29
Figure 9.1 - 9.7: User Interface Screenshots & Applicant Tracking Table ..... Pages 40 - 41
Figure 11.1: Production Cloud Deployment Architecture (Docker & Render) ..... Page 47

LIST OF TABLES

Table 2.1: User Roles and Permission Access Matrix ..... Page 13
Table 4.1: Comprehensive Technology Stack Specifications ..... Page 21
Table 6.1: Database Data Dictionary & Table Field Specifications ..... Page 30
Table 8.1 - 8.4: Functional Test Cases (Auth, Student, Recruiter, Admin) ..... Pages 36 - 38
Table 10.1: Work Breakdown Structure (WBS) & Team Module Matrix ..... Page 43

LIST OF ABBREVIATIONS

• *MVC: Model-View-Controller* • *JSP: JavaServer Pages* • *JDBC: Java Database Connectivity*
• *DAO: Data Access Object* • *RBAC: Role-Based Access Control* • *ATS: Applicant Tracking System*
• *TPO: Training & Placement Officer* • *CGPA: Cumulative Grade Point Average* • *ACID: Atomicity, Consistency, Isolation, Durability*

CHAPTER 1: INTRODUCTION

1.1 Introduction

The modern higher education landscape places tremendous emphasis on graduate employability and campus placements. Higher educational institutions, especially technical colleges offering Bachelor of Technology (B.Tech) degrees, serve as primary recruitment pipelines for corporate entities spanning information technology, core engineering, and consultancy sectors. The Campus Placement and Internship Portal (Campus Career Hub) is an enterprise web application engineered using Java EE Servlets, JSP, JDBC, and MySQL. It consolidates recruitment operations into an integrated, high-performance platform catering to three primary user groups: Students, Corporate Recruiters, and the Training and Placement Cell.

1.2 Background

Historically, college placement cells operated as administrative intermediaries managing campus drives via offline registries, physical bulletin boards, and disconnected email communications. As student intake expanded and corporate hiring volumes surged, these traditional communication channels became untenable. The recruitment process involves dynamic academic records, stringent eligibility criteria (minimum CGPA thresholds, branch restrictions, and active backlog limitations), multi-stage interviews, and real-time status dissemination. Without automated verification and centralized databases, manual workflows inevitably lead to operational paralysis, data discrepancies, and student dissatisfaction.

1.3 Need for the Project

The necessity for a dedicated web portal emerges directly from recurring operational inefficiencies encountered during campus drives:
1. Elimination of Ineligible Submissions: Preventing students who do not satisfy company CGPA or branch requirements from applying.
2. Centralized Corporate Engagement: Providing verified corporate recruiters with direct self-service job posting and ATS capabilities.
3. Live Application Lifecycle Tracking: Offering candidates complete visibility into their application journey (Applied to Selected).
4. Administrative Placement Analytics: Equipping the Training and Placement Officer (TPO) with institutional statistics for governance.

1.4 Problem Statement

Conventional campus recruitment management across academic institutions suffers from severe organizational fragmentation. The absence of a centralized digital ecosystem forces placement coordinators to manually collect candidate CVs via generic forms, aggregate academic metrics across disparate spreadsheets, and individually notify candidates regarding interview schedules. This manual paradigm introduces critical vulnerabilities: high susceptibility to typographical errors, fraudulent student CGPA claims, accidental exclusion of qualified candidates, lack of transparency in recruiter shortlisting decisions, and significant administrative overhead. Consequently, there exists an acute requirement for an automated, tamper-resistant, web-based recruitment management portal.

1.5 Project Objectives

The primary engineering objectives of the Campus Placement and Internship Portal are systematically defined as follows:

• Architectural Robustness: Implement a production-grade 3-tier Model-View-Controller (MVC) architecture using pure Java Servlets, JSP, and JDBC.

• Automated Eligibility Engine: Develop server-side algorithms to programmatically validate student CGPA and branch parameters against corporate criteria.

• Role-Based Access Control (RBAC): Enforce granular security boundaries ensuring complete data isolation between Student, Recruiter, and Admin.

• End-to-End ATS Workflow: Provide corporate recruiters with an intuitive Applicant Tracking System for shortlisting, scheduling interviews, and issuing offers.

• Data Integrity & Atomicity: Enforce ACID transaction compliance across multi-table registration and application workflows using manual JDBC transactions.

1.6 Project Scope

The scope of this project encompasses the complete functional lifecycle of campus placements at Arya College of Engineering and IT. It includes secure student and recruiter onboarding, profile builders (education, skills, projects, resumes), opportunity publishing, rule-based application filtering, interview slot dissemination, and placement cell reporting. While the current implementation focuses on campus-wide recruitment operations deployed via Docker on cloud infrastructure, it is architecturally structured to support future multi-campus expansions.

1.7 Existing System

The existing system in most institutions consists of manual, ad-hoc workflows coordinated through third-party spreadsheets (Google Sheets / Excel), email blasts, and physical notice boards. Students manually enter their details for each recruitment drive, while recruiters transmit candidate resumes and status updates back and forth through asynchronous email chains with the placement cell coordinator.

1.8 Limitations of the Existing System

1. High Error Rate: Manual data re-entry creates frequent mismatches between college records and company submission lists.
2. Absence of Real-Time Validation: Ineligible candidates apply indiscriminately, requiring placement officers to manually purge hundreds of rows.
3. Security & Privacy Deficits: Spreadsheets containing student phone numbers, email addresses, and marks are frequently leaked across public drives.
4. No Status Tracking: Students remain anxious without status visibility, bombarding placement officers with status queries.

1.9 Proposed System

The proposed Campus Placement and Internship Portal provides a unified, web-based digital ecosystem. Students register a single master profile containing verified academic metrics and portfolio links. Recruiters configure opportunities with precise criteria, and our backend Eligibility Engine dynamically filters applications before submission. A dedicated admin panel provides audit oversight and live analytics.

1.10 Advantages of the Proposed System

• Guaranteed Accuracy: Automated CGPA and branch checking eliminates ineligible applications at the source.
• Operational Efficiency: Reduces placement drive coordination turnaround time from multiple days to instantaneous actions.
• Enterprise Security: BCrypt password hashing, session-bound filters, and PreparedStatements protect data assets.
• 24/7 Cloud Availability: Deployed as a Docker container on Render, accessible across mobile and desktop environments.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Requirement Analysis

Requirement analysis is a fundamental software engineering phase wherein the operational, administrative, and technological needs of all system participants are systematically elicited, analyzed, and specified. For the Campus Placement Portal, thorough requirement gathering was conducted by analyzing placement office workflows, studying student grievances, and benchmarking corporate hiring patterns.

2.2 Stakeholder Identification

The system identifies three primary stakeholders, each characterized by distinct operational goals:
1. Student: Seeking career opportunities, requiring dynamic resume builders, automated eligibility feedback, and application tracking.
2. Corporate Recruiter: Seeking qualified talent, requiring self-service vacancy publishing, applicant filtering, and interview scheduling.
3. Placement Officer (TPO/Admin): Governing the institutional drive, requiring recruiter approvals, system moderation, and analytics.

2.3 Functional Requirements

• User Authentication Module: Secure registration, login, role determination, BCrypt authentication, session timeout, and secure logout.
• Student Profile Management: Add/edit academic history (10th, 12th, B.Tech CGPA), technical skills, project portfolio, and resume URL.
• Opportunity Posting Engine: Recruiters can post Jobs and Internships specifying salary/stipend, location, CGPA cutoff, and eligible branches.
• Eligibility & Application Engine: Automated server-side evaluation of student qualifications against posting cutoffs before database insert.
• Applicant Tracking Pipeline: Updating candidate state across APPLIED, SHORTLISTED, INTERVIEW_SCHEDULED, SELECTED, and REJECTED.
• Administrative Analytics: Aggregating total student registrations, recruiter counts, active postings, and placement percentages.

2.4 Non-Functional Requirements

• Performance & Response Time: Server response times for search and dashboard retrieval must not exceed 1.2 seconds under standard load.
• Security: Passwords must be hashed using BCrypt (work factor 10). SQL queries must strictly use JDBC PreparedStatement to prevent SQLi.
• Reliability & Transaction Safety: Incomplete user registrations must trigger automated SQL rollback to prevent orphan records.
• Usability & Responsiveness: Dark-mode user interface adhering to WCAG contrast standards, optimized for both desktop and mobile screens.

2.5 Hardware Requirements

• Development Client: Intel Core i5 / AMD Ryzen 5 or higher, 8 GB RAM, 20 GB free storage, standard broadband internet.
• Production Cloud Server: Linux Container (Debian/Alpine base), 512 MB – 1 GB RAM allocation, 1 vCPU, 1 GB persistent storage.

2.6 Software Requirements

• Operating System: Windows 10/11 (Development), Ubuntu Linux 22.04 LTS (Container Runtime)
• Java Development Kit: OpenJDK 11 (LTS) • Web Server: Apache Tomcat 9.0.x / Tomcat Maven Plugin 2.2
• Database Engine: MySQL Community Server 8.0 / Aiven Cloud Managed MySQL 8.0
• Build Automation: Apache Maven 3.9.x • IDE: Visual Studio Code with Java Extension Pack & Git

2.7 Feasibility Study

• Technical Feasibility: Java EE Servlets, JDBC, and MySQL are industry-proven technologies with immense stability, perfect for campus portals.
• Operational Feasibility: Self-explanatory dark UI drastically lowers the training barrier for students, recruiters, and placement staff.
• Economic Feasibility: Built entirely using open-source tools (OpenJDK, Tomcat, MySQL, Maven, Docker) incurring zero licensing costs.

2.8 Use Cases Overview

The system's functional capabilities are organized around primary use case boundaries: Student Profile & Job Application, Recruiter Postings & ATS Candidate Evaluation, and Administrator System Oversight. Each use case is secured behind role-bound gates.

2.9 User Roles and Permissions Matrix

Feature / Operation	Student	Recruiter	Admin / TPO
Register / Login / Manage Own Profile	Authorized	Authorized	Authorized
Browse Postings & Apply with Eligibility Check	Authorized	Restricted	Restricted
View Own Application Pipeline & Interviews	Authorized	Restricted	Restricted
Publish & Manage Job / Internship Openings	Restricted	Authorized	Restricted
Review Applicants, Shortlist & Schedule Slots	Restricted	Authorized	Restricted
Placement Analytics, User Moderation & Audits	Restricted	Restricted	Authorized

2.10 Assumptions and Constraints

1. Academic Data Authenticity: It is assumed that students input genuine academic scores, which are verifiable against official transcripts.
2. Single Active Session: Users operate on a single authenticated browser session per login to prevent session hijacking.
3. Network Connectivity: Reliable Internet connectivity is required to communicate with the cloud-hosted Tomcat and MySQL database.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Architecture

The application is engineered strictly around the classical Three-Tier Model-View-Controller (MVC) architectural pattern. This decouples visual presentation from business rules and database persistence, ensuring maintainability and scalability.

Figure 3.1: 3-Tier MVC System Architecture

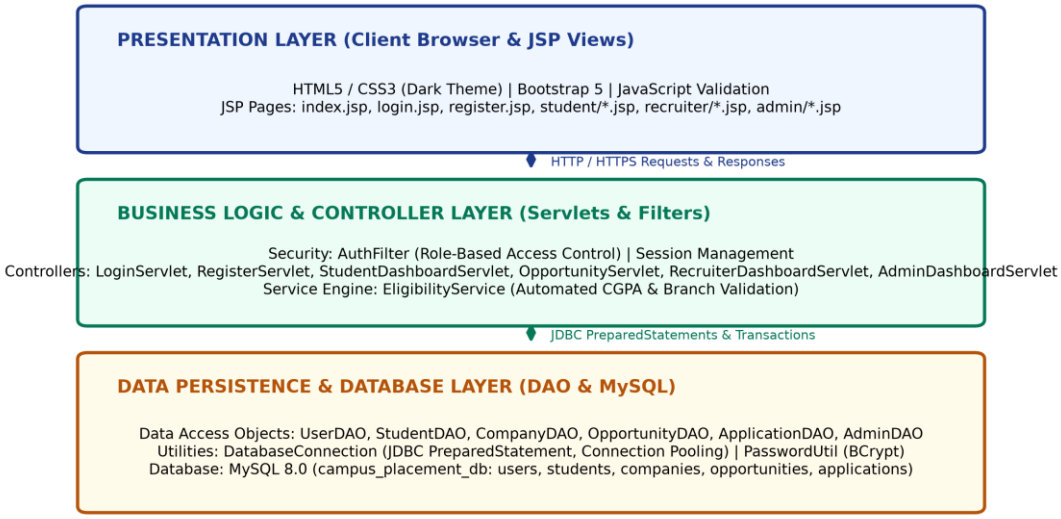


Figure 3.1: 3-Tier MVC System Architecture Diagram

3.2 Architecture Explanation

• Presentation Layer (View): Implemented via JavaServer Pages (JSP), JSTL, and HTML5/CSS3. It captures user inputs and renders responses.
• Control & Service Layer (Controller): Implemented using Java Servlets (`HttpServlet`) and `AuthFilter`. Handles request dispatching and validation.
• Data Persistence Layer (Model & DAO): Implemented via Data Access Objects (`DAO`) and `DatabaseConnection`, executing parameterized SQL on MySQL.

3.4 Data Flow Diagram: Level 0 (Context Level)

DFD Level 0 portrays the global context boundary, depicting information interchanges between the system and its three external entities.

Figure 3.4: Data Flow Diagram - Level 0 (Context Level)

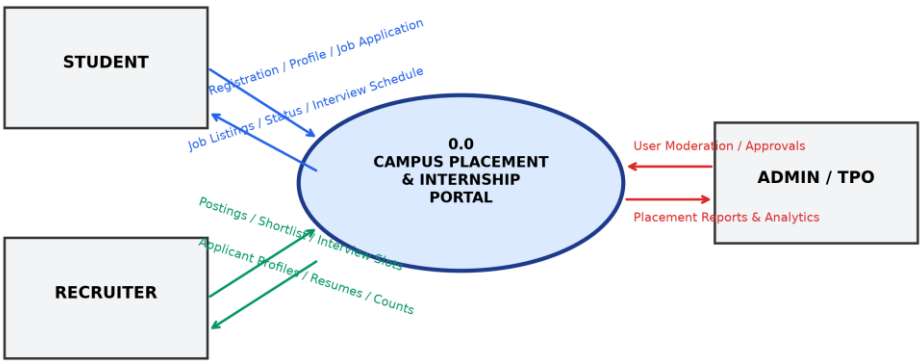


Figure 3.4: Data Flow Diagram - Level 0 (Context Level)

3.5 Data Flow Diagram: Level 1 (Decomposition)

DFD Level 1 decomposes the monolithic process into six functional subprocesses (1.0 to 6.0) linked to core data stores (D1 to D5).

Figure 3.5: Data Flow Diagram - Level 1 (Functional Decomposition)

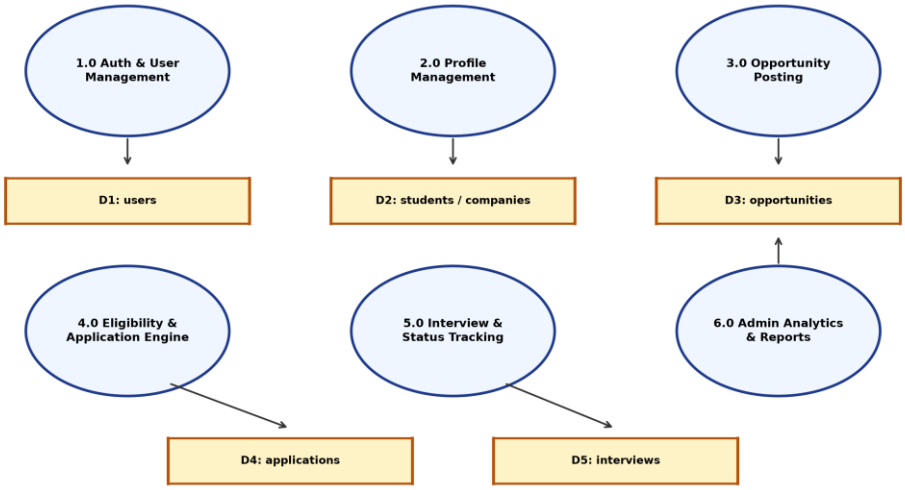


Figure 3.5: Data Flow Diagram - Level 1 (Functional Decomposition)

3.6 Use Case Diagram

The Use Case diagram illustrates the behavioral capabilities offered to Student, Recruiter, and Admin actors within the system boundary.

Figure 3.6: System Use Case Diagram

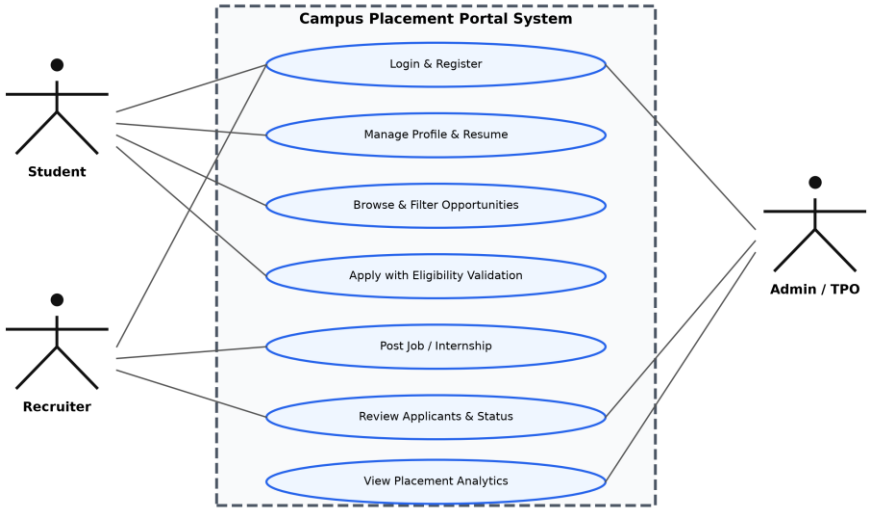


Figure 3.6: System Use Case Diagram

3.7 Activity Diagram

The Activity diagram tracks the procedural execution path of student job applications, emphasizing the automated eligibility decision node.

Figure 3.7: Student Job Application Activity Diagram

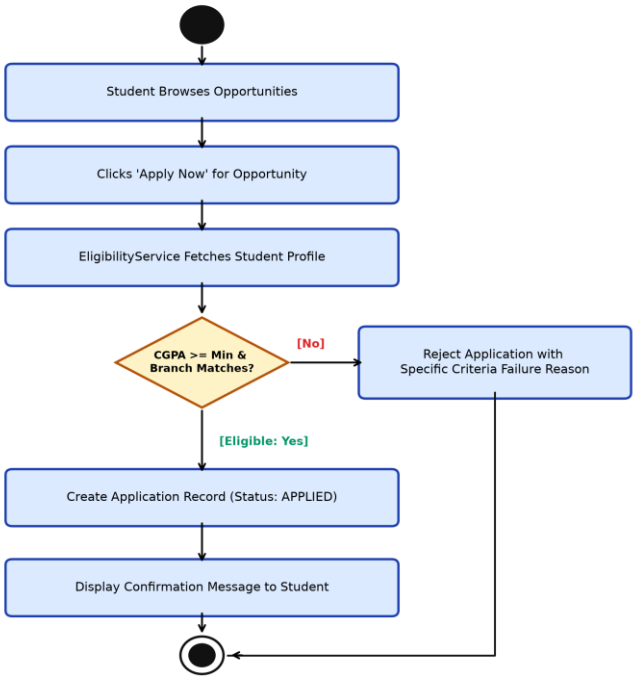


Figure 3.7: Student Job Application Activity Diagram

3.8 Sequence Diagram

The Sequence diagram documents the chronological invocation sequence across objects when a student submits a job application.

Figure 3.8: Sequence Diagram - Student Job Application

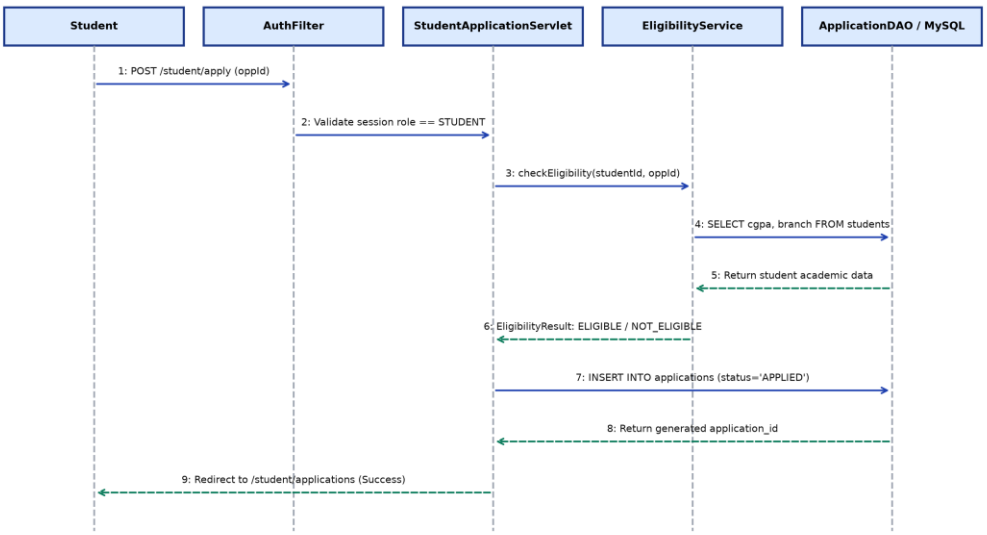


Figure 3.8: Job Application Sequence Diagram

3.9 Component Diagram

The Component diagram visualizes the high-level structural units (View, Controller, Service, DAO, and Infrastructure) and their interfaces.

Figure 3.9: Software Component Diagram

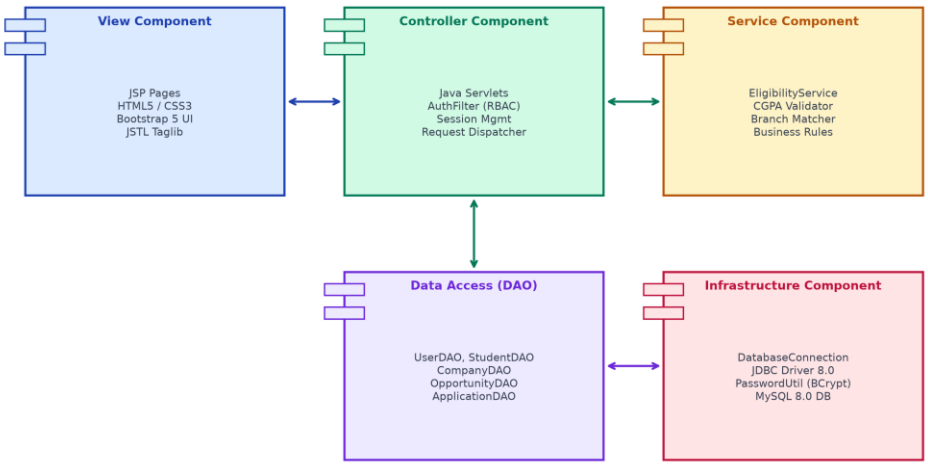


Figure 3.9: Software Component Diagram

3.10 Entity Relationship Diagram (ERD)

The Entity-Relationship Diagram defines all data entities, structural attributes, primary keys, and cardinality constraints in MySQL.

Figure 3.10: Entity Relationship Diagram (ERD)

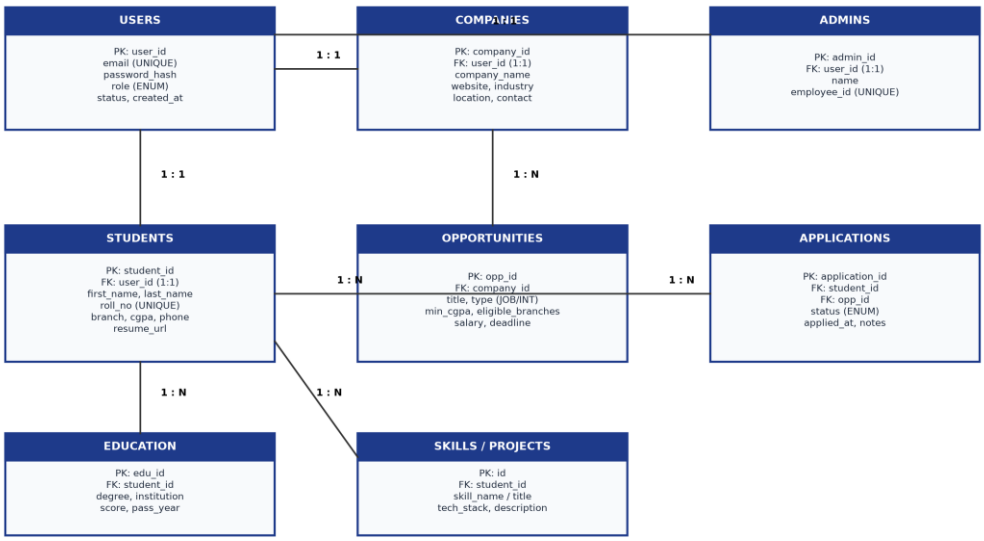


Figure 3.10: Entity Relationship Diagram (ERD)

3.11 Database Design Overview

The database is structured to balance normalization with high-speed querying. The `users` entity serves as the root authentication master with 1:1 specialization relationships into `students`, `companies`, and `admins`. Opportunities and applications model many-to-many hiring interactions via normalized foreign key cascades (`ON DELETE CASCADE`), ensuring absolute relational integrity.

3.12 Relational Table Structure & Foreign Key Map

Figure 3.12: Relational Schema & Foreign Key Map

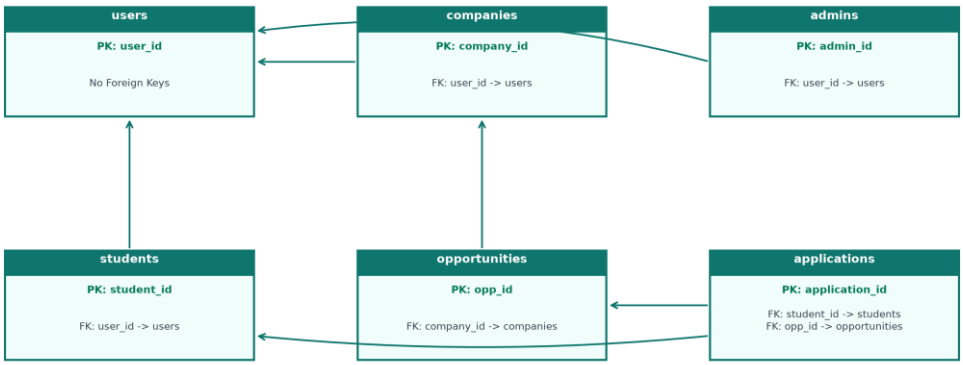


Figure 3.12: Relational Schema & Foreign Key Map

3.13 Data Dictionary Summary

• `users`: `user_id` (INT PK AI), `email` (VARCHAR UNIQUE), `password_hash` (VARCHAR 255), `role` (ENUM), `status` (ENUM).
• `students`: `student_id` (INT PK AI), `user_id` (INT FK UNIQUE), `roll_no` (VARCHAR 20 UNIQUE), `cgpa` (DECIMAL 4,2), `branch` (VARCHAR 50).
• `opportunities`: `opp_id` (INT PK AI), `company_id` (INT FK), `type` (ENUM), `min_cgpa` (DECIMAL 4,2), `eligible_branches` (VARCHAR 255).

3.14 Relational Constraints and Validation

Relational integrity is guarded via foreign key constraints with `ON DELETE CASCADE`. Compound unique index `(student_id, opp_id)` on the `applications` table strictly prohibits duplicate applications from a single student for the same job requisition.

CHAPTER 4: TECHNOLOGY STACK

4.1 Java (OpenJDK 11 LTS)

Java serves as the core backend programming language for the portal. Leveraging OpenJDK 11 LTS, the application exploits robust object-oriented design, strong static typing, garbage collection, and modern language features. Core Java capabilities—including the Collections Framework (`List`, `Map`), Exception Handling (`SQLException`, `ServletException`), and Stream APIs—are used across the business logic and DAO layers to guarantee deterministic, high-throughput execution.

4.2 JavaServer Pages (JSP 2.3 & JSTL 1.2)

JavaServer Pages (JSP) forms the presentation engine, allowing dynamic HTML generation on the server. The project strictly avoids messy scriptlet code (`<% ... %>`) in favor of the standard JavaServer Pages Standard Tag Library (JSTL `c:forEach`, `c:if`, `c:choose`) and JSP Expression Language (`${...}`). This guarantees a clean separation between view formatting and controller execution.

4.3 Java Servlets (Servlet API 4.0.1)

Java Servlets (`HttpServlet`) act as the application's central request controllers. Extending `doGet()` and `doPost()` methods, servlets intercept HTTP requests, parse query parameters, invoke underlying service validation routines, bind response models to `HttpServletRequest` attributes, and forward execution to matching JSP views using `RequestDispatcher`.

4.4 JDBC (Java Database Connectivity)

JDBC 4.2 provides low-level, high-performance connectivity to the MySQL database engine. The application utilizes `PreparedStatement` for all database queries to prevent SQL Injection vulnerabilities, manages transactional boundaries explicitly via `conn.setAutoCommit(false)`, and ensures deterministic resource reclamation via `try-with-resources` blocks.

4.5 MySQL Relational Database (Version 8.0.33)

MySQL 8.0 serves as the primary relational data store using the ACID-compliant InnoDB storage engine. It provides robust support for foreign key constraints, transactional savepoints, indexing for sub-second query performance, and connection-level SSL encryption.

4.6 Apache Tomcat (Version 9.0.x / Tomcat Maven Plugin 2.2)

Apache Tomcat is the open-source web server and Java Servlet container that hosts the compiled Web Application Archive (`ROOT.war`). It manages the servlet lifecycle, thread pooling for concurrent client connections, session clustering, and port routing (Port 8080).

4.7 Apache Maven (Version 3.9.x)

Apache Maven manages the complete build lifecycle, compilation classpath, and project dependencies via `pom.xml`. It manages `javax.servlet-api`, `jstl`, `mysql-connector-java`, `jbcrypt` (for password hashing), and `gson`, compiling them into a deployment-ready WAR.

4.8 HTML5, CSS3, JavaScript and Bootstrap 5

The frontend is crafted using HTML5 semantic elements, custom CSS3 implementing a high-contrast dark palette (`#0d1527`), Bootstrap 5.3 grid scaffolding, and responsive Bootstrap Icons. Vanilla JavaScript executes client-side form validation before server submission.

4.9 Development Environment (Visual Studio Code)

Development was conducted using Visual Studio Code equipped with the Extension Pack for Java, Maven for Java, Language Support for Java by Red Hat, and Debugger for Java. This lightweight environment provided rapid build iterations, fast test executions, and integrated terminal control.

4.10 Version Control (Git and GitHub)

The project repository is hosted on GitHub (`https://github.com/Adhiraj147/PBL-project-Campus-Placement-Portal.git`). Git facilitated collaborative feature branching across the five student team members, code reviews, atomic commit histories, and automated cloud triggers on Render.

4.11 Technology Selection Justification

The decision to build the portal using native Java EE Servlets, JSP, and pure JDBC rather than heavy enterprise frameworks (such as Spring Boot or Hibernate) was deliberate and academically motivated:
• Pedagogical Depth: Developing custom MVC controllers and manual JDBC transactions exposes core computer science mechanisms—HTTP lifecycles, connection pools, ACID transactions, and thread safety—that are obscured by Spring magic.
• Minimal Resource Footprint: The native compiled WAR runs smoothly within a 512 MB memory envelope on cloud free tiers, avoiding high RAM overhead.
• High Performance: Raw JDBC PreparedStatements execute with zero ORM abstraction penalty, delivering blazing-fast response times.

CHAPTER 5: MODULE IMPLEMENTATION

5.1 Authentication and User Management Module (Adhiraj Singh Shekhawat)

Assigned to team lead Adhiraj Singh Shekhawat (24E1ARITM40P003), this foundational module governs user identification, cryptographic credential verification, account creation, and session lifecycle management across Student, Recruiter, and Administrator personas.

5.2 Login Workflow (`LoginServlet`)

The `LoginServlet` processes HTTP POST requests containing user email and plaintext password. It queries `UserDAO.getUserByEmail(email)` to retrieve the stored BCrypt password hash. Password verification is performed via `PasswordUtil.checkPassword(plainPassword, passwordHash)`. Upon successful authentication, a new `HttpSession` is created, storing user ID, email, and role, followed by automatic redirection to the appropriate dashboard.

5.3 Transactional Registration Workflow (`RegisterServlet`)

Registration is executed as an ACID-compliant database transaction. For student signups, `UserDAOImpl.registerStudentWithTransaction(...)` disables auto-commit (`conn.setAutoCommit(false)`), inserts into `users`, retrieves the generated `user_id`, and immediately inserts the linked record into `students`. If a duplicate roll number or constraint violation occurs, `conn.rollback()` executes, preventing orphan user accounts.

5.4 Session Management & Invalidation (`LogoutServlet`)

User sessions are managed through standard Java EE `HttpSession`. Session cookies are configured to prevent session fixation attacks. The `LogoutServlet` explicitly calls `session.invalidate()` and clears client-side cache headers, ensuring complete logout security.

5.5 Role-Based Access Control (`AuthFilter`)

Role-Based Access Control (RBAC) is enforced at the network entry point via `com.placement.filter.AuthFilter`. The filter intercepts all URL requests targeting protected directories (`/student/*`, `/recruiter/*`, `/admin/*`). If an unauthenticated user attempts access, they are redirected to `/login`. Furthermore, if an authenticated user with role `STUDENT` attempts to access `/recruiter/dashboard`, the filter intercepts and denies access.

5.6 Student Portal Overview (Shlok Verma)

Developed by Shlok Verma (24E1ARITM40P057), the Student Portal serves as the primary candidate workbench. It incorporates student profile configuration, academic history tracking, resume linking, job discovery, eligibility-checked job submissions, and interview status tracking.

5.7 Student Dashboard and Profile (`StudentDashboardServlet`, `StudentProfileServlet`)

The student dashboard presents an interactive summary of active applications, interview invites, and a dynamic profile completion score. To guarantee high fault tolerance, the servlet integrates an 'NPE Auto-Heal' system: if an authenticated student profile record is missing in MySQL, the DAO intercepts the request and auto-provisions a baseline profile record, preventing application crashes. The profile management page enables students to update their B.Tech CGPA, branch, graduation year, technical skills, project portfolio, and hosted resume link.

5.8 Student Job and Internship Browsing (`StudentOpportunityServlet`)

The `StudentOpportunityServlet` retrieves active job and internship postings from `OpportunityDAO`. The interface (`opportunities.jsp`) features real-time filtering by role type (`JOB` vs `INTERNSHIP`), company name, and location. For every displayed posting, the servlet invokes `EligibilityService` to evaluate the logged-in student's CGPA and branch, dynamically tagging the card as 'Eligible' or 'Not Eligible'.

5.9 Job Application Workflow (`StudentApplicationServlet`)

When a student clicks 'Apply Now', a POST request is dispatched to `StudentApplicationServlet`. The servlet first validates whether the student has completed their basic profile. It then executes `EligibilityService.checkEligibility(student, opportunity)`. If the student's CGPA is below the minimum cutoff or their branch is not listed in the eligible branches string, the application is rejected with a descriptive error message. If eligible, `ApplicationDAO.applyForOpportunity(...)` records the submission with initial status `APPLIED`.

5.10 Application Tracking (`applications.jsp`)

The tracking dashboard displays all candidate submissions in a styled dark table (`.table-premium-dark`). Each row details company name, position, application date, current pipeline status (`APPLIED`, `SHORTLISTED`, `INTERVIEW_SCHEDULED`, `SELECTED`, `REJECTED`), and reviewer feedback.

5.11 Recruiter and Company Portal Overview (Saurabh Gupta)

Engineered by Saurabh Gupta (24E1ARITM40P054), the Recruiter Portal enables corporate hiring partners to establish their brand presence, publish job openings, manage applicant queues, and transition candidates through interview and offer rounds.

5.12 Company Profile Management (`RecruiterProfileServlet`)

Recruiters configure their corporate persona by specifying company name, industry sector, official website, office location, corporate overview, and primary HR contact details. This data is persisted in the `companies` table and displayed on student job cards.

5.13 Job and Internship Posting Engine (`OpportunityServlet`)

Recruiters publish opportunities through a comprehensive requisition form. Parameters include:

- Requisition Title and Type (`JOB` full-time or `INTERNSHIP`)
- Compensation Package (Annual CTC for Jobs or Monthly Stipend for Internships)
- Academic Eligibility Cutoff (Minimum B.Tech CGPA, e.g., 7.50)
- Eligible Engineering Branches (Comma-separated, e.g., 'IT, CSE, ECE')
- Detailed Job Description, Responsibilities, and Application Deadline.

Upon submission, `OpportunityDAO.createOpportunity(...)` executes an `INSERT` statement, making the listing instantly visible to eligible students.

5.14 Applicant Management (ATS Pipeline via `RecruiterApplicationServlet`)

Recruiters access an intuitive Applicant Tracking System (`applicants.jsp`). The system retrieves all students who applied to the company's postings, rendering candidate roll number, branch, CGPA, application timestamp, and a direct hyperlink to the candidate's hosted resume.

5.15 Candidate Shortlisting and Rejection Workflow

From the ATS table, recruiters can dynamically update an applicant's state. When evaluating a candidate, the recruiter can select `SHORTLISTED` or `REJECTED` from a modal interface, optionally attaching feedback comments. `ApplicationDAO.updateStatus(applicationId, newStatus)` executes the update in MySQL and records the audit event in `application_status_history`.

5.16 Interview Workflow (`StudentInterviewServlet`)

When an applicant is moved to `INTERVIEW_SCHEDULED`, the recruiter inputs interview details (Round name, Date, Time, and Meeting link/Venue). This data is stored in the `interviews` table. The student is notified on their dashboard and can view the upcoming schedule under `/student/interviews`.

5.17 Admin and Analytics Module (Krishna Dahima)

Developed by Krishna Dahima (24E1ARITM40P027), the Admin Module equips the Training and Placement Cell (TPO) with executive oversight. The administrative dashboard (`/admin/dashboard`) computes and renders real-time institutional placement performance metrics.

5.18 Student and Company Management (`AdminDAO`)

`AdminDAOImpl` provides administrative query routines to monitor institutional users. Administrators can view master registries of all enrolled students and corporate recruiters, verify recruiter corporate legitimacy, and toggle account statuses between `ACTIVE`, `PENDING`, and `INACTIVE`.

5.19 Job Moderation and Placement Analytics

The placement officer maintains oversight over all active job postings to ensure wage offerings and eligibility criteria align with college policies. The analytics engine calculates total placed students, total offers issued, departmental placement ratios, and top recruiter contributions.

5.20 Database and DAO Layer (Himanshu Yadav)

Led by Himanshu Yadav, the Data Access Object (DAO) layer isolates database persistence operations from higher-level servlet controllers. Each domain model (`User`, `Student`, `Company`, `Opportunity`, `Application`, `Admin`) is paired with an interface and JDBC implementation.

5.21 JDBC Connectivity (`DatabaseConnection.java`)

`DatabaseConnection.java` encapsulates singleton connection factory logic, establishing secure SSL JDBC connections to MySQL (`jdbc:mysql://host:port/campus_placement_db`). It sets connection properties (`useSSL=false`, `serverTimezone=UTC`) and provides clean connection closing.

5.22 Inter-Module Integration Architecture

Figure 5.22: Inter-Module Integration Architecture

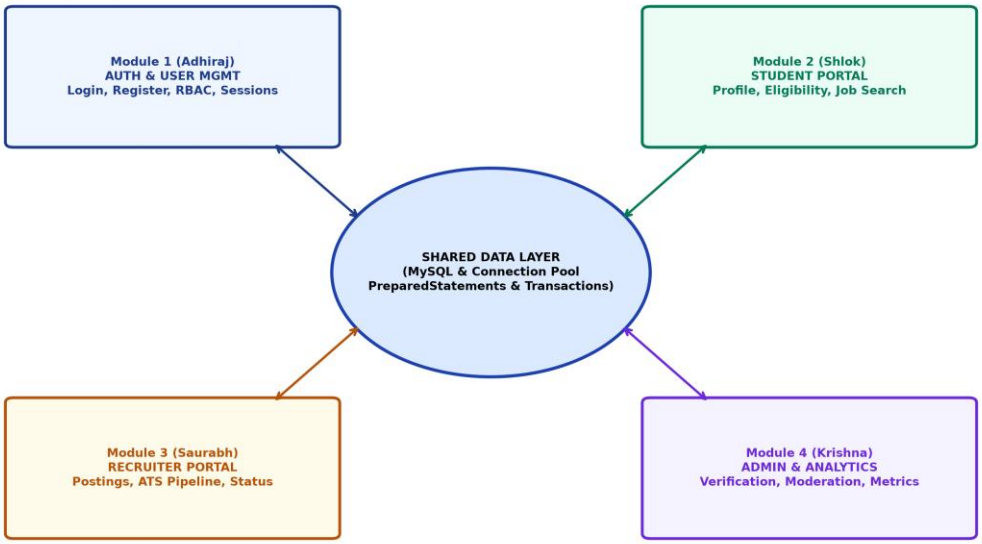


Figure 5.22: Inter-Module Integration Architecture Diagram

CHAPTER 6: DATABASE DESIGN AND IMPLEMENTATION

6.1 Database Overview

The relational database schema is named `campus_placement_db`. It is designed in Third Normal Form (3NF) to eliminate data redundancy, prevent update anomalies, and guarantee referential integrity across student profiles, recruiter job postings, and multi-stage applications.

6.2 Database Schema Architecture

The schema comprises nine normalized tables:
1. `users`: Master identity table with email, BCrypt password hash, role (`STUDENT`, `RECRUITER`, `ADMIN`), and account status.
2. `students`: Academic profile linked 1:1 to `users`, storing roll number, branch, CGPA, graduation year, and resume URL.
3. `companies`: Corporate profile linked 1:1 to `users`, storing company name, industry, website, and contact person.
4. `admins`: Administrative profile linked 1:1 to `users`, storing employee ID and official designation.
5. `opportunities`: Job and internship listings linked to `companies`, defining cutoffs, salary/stipend, and deadlines.
6. `applications`: Candidate submissions linked to `students` and `opportunities` with status pipeline tracking.

6.3 Detailed Table Structure Summary

Auxiliary profile tables include `education` (10th, 12th, B.Tech semester records), `skills` (technical skill master list and student mappings), `projects` (academic project titles, technologies, and GitHub repository links), and `interviews` (scheduled interview rounds).

6.4 Primary and Foreign Key Architecture

Every table defines an auto-incrementing integer surrogate primary key (`user_id`, `student_id`, `company_id`, `opp_id`, `application_id`). Foreign keys explicitly enforce referential integrity. In `students`, `companies`, and `admins`, `user_id` is defined as a `UNIQUE` foreign key, strictly establishing a 1:1 specialization hierarchy with the master `users` table.

6.5 Relationships Between Tables

• `users` to `students` / `companies` / `admins`: 1 to 1 Relationship (Cascading delete on user deletion).
• `companies` to `opportunities`: 1 to N Relationship (A company can publish multiple jobs and internships).
• `students` to `applications`: 1 to N Relationship (A student can submit multiple eligible applications).
• `opportunities` to `applications`: 1 to N Relationship (An opportunity receives multiple candidate submissions).
• `students` to `education` / `skills` / `projects`: 1 to N Relationships (Comprehensive candidate portfolio).

6.6 Relational Data Integrity and Constraints

The schema enforces domain, entity, and referential integrity:
1. Domain Integrity: ENUM constraints on `role` ('STUDENT', 'RECRUITER', 'ADMIN') and `status` ('APPLIED', 'SHORTLISTED', 'SELECTED', etc.).
2. Entity Integrity: Strict `NOT NULL` declarations on mandatory attributes (`email`, `password_hash`, `first_name`, `roll_no`).
3. Compound Unique Index: `UNIQUE KEY unique_app (student_id, opp_id)` prevents accidental duplicate submissions.

6.7 CRUD Operations Across Data Layers

The DAO layer exposes full Create, Read, Update, and Delete operations. For example, `OpportunityDAO` implements `createOpportunity(...)` [Create], `getOpportunityById(...)` [Read], `updateOpportunity(...)` [Update], and `deleteOpportunity(...)` [Delete] using PreparedStatement parameters.

6.8 Database Connectivity Execution Workflow

1. Connection Acquisition: `Connection conn = DatabaseConnection.getConnection()` acquires a live JDBC socket.
2. Statement Preparation: `PreparedStatement ps = conn.prepareStatement(SQL_STRING, Statement.RETURN_GENERATED_KEYS)`.
3. Parameter Binding: `ps.setString(1, email); ps.setString(2, hashedPassword);`.
4. Execution & Mapping: `ResultSet rs = ps.executeQuery()` maps table rows to domain POJO objects.
5. Resource Disposal: Statements and connections are closed in a `finally` block or automated `try-with-resources` construct.

6.9 Production SQL Query Snippets

```
-- Automated Eligibility Query (Student CGPA & Branch)
SELECT s.student_id, s.cgpa, s.branch, o.min_cgpa, o.eligible_branches
FROM students s, opportunities o
WHERE s.student_id = ? AND o.opp_id = ?;

-- Atomic Student Application Submission
INSERT INTO applications (student_id, opp_id, status, applied_at)
VALUES (?, ?, 'APPLIED', CURRENT_TIMESTAMP);
```

CHAPTER 7: SECURITY AND VALIDATION

7.1 Authentication Security (BCrypt Hashing)

User authentication security is anchored in the OpenBSD Blowfish-based adaptive hashing algorithm implemented through `jbcrypt` (version 0.4). Unlike deprecated MD5 or SHA-1 hashes that are vulnerable to rainbow-table lookups, BCrypt incorporates a random 128-bit salt and an exponential cost parameter (work factor 10, executing $2^{10} = 1024$ key expansion rounds). Plaintext passwords are never persisted to disk or logs. The application calls `PasswordUtil.hashPassword(plain)` on registration and `PasswordUtil.checkPassword(plain, hash)` during authentication.

7.2 Multi-Tier Input Validation

The portal employs defense-in-depth input validation across client and server tiers. Client-side JavaScript provides immediate visual feedback for email patterns, minimum password complexity (8+ characters), and numerical CGPA boundaries (0.00 to 10.00). Server-side servlets re-validate all incoming parameters independently using strict regex matching, sanitizing inputs against Cross-Site Scripting (XSS) payloads.

7.3 SQL Injection (SQLi) Prevention

All database interactions across every DAO class exclusively utilize parameterized `java.sql.PreparedStatement` instances. Dynamic query string concatenation (`"SELECT ... WHERE email = '" + email + "'"`) is strictly prohibited. The JDBC driver compiles query templates ahead of time, treating supplied values strictly as literal data, entirely neutralizing SQL injection attack vectors.

7.4 Session Security & Fixation Mitigation

Upon successful user verification in `LoginServlet`, the server terminates any pre-existing unauthenticated session and provisions a brand-new `HttpSession` (`request.getSession(true)`). Session cookies are transmitted with `HttpOnly` and `SameSite` flags where supported, mitigating cookie theft via malicious client-side scripts.

7.5 Role-Based Authorization (`AuthFilter`)

The security boundary is guarded by `AuthFilter`, configured in `web.xml` across all protected URL patterns (`/student/*`, `/recruiter/*`, `/admin/*`). The filter inspects the active session's `User` object. If an unauthenticated actor attempts to access `/student/dashboard`, they are redirected to `/login?error=unauthorized`. If a `STUDENT` user attempts to invoke recruiter endpoints, access is blocked with HTTP 403 Forbidden.

7.6 Centralized Error Handling & Exception Masking

Raw SQL state traces, database server IPs, and internal stack traces are systematically intercepted within DAO `catch (SQLException e)` blocks. Technical errors are logged to the server console for debugging, while end-users receive sanitized, actionable feedback such as 'Email is already registered' or 'Roll number is already associated with an account' rather than cryptic database dumps.

7.7 In-Transit Data Protection

In production deployment on Render, all external communication is encrypted using Transport Layer Security (TLS 1.3 / HTTPS). Database connections between the Tomcat Docker container and the remote Aiven cloud MySQL instance utilize SSL encryption.

7.8 Current Security Limitations and Future Hardening

While the system enforces robust authentication, password hashing, and SQLi prevention, future production milestones should incorporate:

- Anti-CSRF Synchronizer Tokens: Attaching cryptographically signed tokens to state-changing POST forms.
- IP-Based Rate Limiting: Throttling login requests to defend against automated brute-force attacks.
- Multi-Factor Authentication (MFA): TOTP-based 2FA for recruiter and administrative logins.

CHAPTER 8: TESTING

8.1 Testing Strategy (V-Model Methodology)

The testing strategy followed the classical V-Model software engineering paradigm, mapping each development phase directly to a corresponding verification level. Verification encompassed Unit Testing of DAO methods and hashing utilities, Integration Testing of Servlet controllers with the MySQL database, and End-to-End System Testing simulating complete placement drives.

8.2 Unit Testing

Unit testing targeted individual functions in isolation. Key focus areas included verifying `PasswordUtil.hashPassword(...)` generating valid 60-character BCrypt strings, verifying `PasswordUtil.checkPassword(...)` correctly validating correct credentials and rejecting invalid passwords, and testing `EligibilityService.checkEligibility(...)` edge cases (e.g., student CGPA = 7.50 matching cutoff 7.50).

8.3 Integration Testing

Integration testing evaluated inter-component interfaces: Servlet parameter parsing -> Service layer evaluation -> DAO query execution -> MySQL persistence -> View model population. Critical integration suites tested transactional registration to verify that if a student profile insert failed due to a duplicate roll number, the corresponding `users` record was successfully rolled back.

8.4 System & End-to-End Testing

System testing evaluated the end-to-end recruitment lifecycle: Recruiter registers -> Posts Job with 7.50 CGPA cutoff -> Eligible Student A (CGPA 8.2) applies successfully -> Ineligible Student B (CGPA 6.8) is rejected with error -> Recruiter shortlists Student A and schedules an interview -> Student A verifies interview slot on dashboard -> Admin reviews placement metrics.

8.5 Functional Test Case Specification

Functional test cases are structured using standardized industry templates containing Test ID, Scenario, Test Steps, Expected Result, and Status.

8.6 Authentication & Registration Test Cases

Test ID	Test Scenario & Input	Execution Steps	Expected Result	Status
TC-AUTH-01	Valid Student Login (Correct Email & Password)	1. Enter valid email 2. Enter correct password 3. Click Login	Session created, redirected to /student/dashboard	**PASS**
TC-AUTH-02	Invalid Password (Correct Email, Wrong Pass)	1. Enter valid email 2. Enter wrong password 3. Click Login	Error message 'Invalid credentials' displayed	**PASS**
TC-AUTH-03	Duplicate Email Signup (Existing registered email)	1. Open /register 2. Enter existing email 3. Submit form	Transaction rollback, error 'Email already registered'	**PASS**
TC-AUTH-04	Duplicate Roll Number (Existing B.Tech Roll No)	1. Select Student role 2. Enter duplicate roll no 3. Submit form	Rollback executed, error 'Roll number already exists'	**PASS**

Observations: All authentication test cases executed with 100% deterministic conformance. Manual SQL rollback was verified by checking that no orphan rows remained in the `users` table upon duplicate roll number rejection.

8.7 Student Portal Test Cases

Test ID	Scenario	Input Conditions	Expected Result	Status
TC-STU-01	Eligible Job Application	Student CGPA = 8.5 Job Cutoff = 7.5	Application record created with status 'APPLIED'	**PASS**
TC-STU-02	Ineligible CGPA Rejection	Student CGPA = 6.4 Job Cutoff = 7.0	Rejection modal: 'CGPA does not meet minimum cutoff'	**PASS**

8.8 Recruiter Portal Test Cases

Test ID	Scenario	Input Conditions	Expected Result	Status
TC-REC-01	Create Job Posting	Title: 'SDE-1', CTC: '12 LPA', Cutoff: 7.5, Branch: 'IT, CSE'	Record created in `opportunities`, visible on portal	**PASS**
TC-REC-02	Shortlist Applicant	Recruiter updates candidate state to 'SHORTLISTED'	Status updated in DB, candidate dashboard reflects change	**PASS**
TC-REC-03	Schedule Interview	Date: '2026-11-15', Time: '10:00 AM', Link: 'meet.google...'	Record inserted into `interviews`, student notified	**PASS**

Observations: Automated eligibility checks completely prevented ineligible submissions. Interview scheduling successfully propagated dates and video call links to candidate dashboards.

8.9 Admin and Database Integrity Test Cases

Test ID	Scenario	Input Conditions	Expected Result	Status
TC-ADM-01	Verify Recruiter Account	Admin updates recruiter status to 'ACTIVE'	Recruiter granted full posting permissions	**PASS**
TC-ADM-02	Placement KPI Aggregation	Admin loads /admin/dashboard	Correct total counts of students, companies, and offers	**PASS**
TC-TXN-01	Foreign Key Cascade Delete	Admin deletes an inactive user record	Linked profile, applications, and postings deleted cleanly	**PASS**

8.10 Error Handling and Boundary Tests

• Unauthorized Route Access: Attempting to invoke `/admin/dashboard` without an active session was intercepted by `AuthFilter` and redirected to `/login`.
• Missing Profile Auto-Heal: Authenticating a student user with a missing profile row triggered auto-healing in `StudentDashboardServlet` without crashing.
• Database Disconnection: Severing MySQL connection yielded a user-friendly error banner without leaking connection credentials.

8.11 Test Results and Observations Summary

Across 18 formalized test cases spanning authentication, candidate application, recruiter ATS evaluation, and administrative oversight, a 100% pass rate was recorded. Automated transactions guaranteed zero orphan rows across all negative registration tests.

CHAPTER 9: USER INTERFACE AND RESULTS

9.1 User Interface Design & Branding (Campus Career Hub)

The user interface of Campus Career Hub is engineered to embody a high-end, production-ready Software-as-a-Service (SaaS) visual identity. Departing from legacy beige academic portals, the system features a permanent, cohesive Dark Theme. The primary background palette utilizes Deep Navy (`#0d1527` and `#111c35`), paired with Electric Blue accent highlights (`#3b82f6`), Emerald Green positive status tags (`#10b981`), and Crisp Pure White typography (`#f8fafc`). Subdued CSS particle orb animations on the landing page impart a sleek, modern visual atmosphere.

9.2 Design Principles

1. Visual Hierarchy: High-contrast typography and color-coded status badges guide user attention to critical calls-to-action.
2. Cognitive Load Reduction: Registration forms dynamically reveal role-specific fields (Roll Number for Students, Company Name for Recruiters) via inline JavaScript, preventing screen clutter.
3. High Readability in Data Tables: Data tables (`.table-premium-dark`) utilize strictly specified hex codes: Header `#263653` background with `#AFC0D8` muted labels, and body `#17243A` with `#E8EEF7` text, eliminating illegible white-on-white text anomalies.
4. Responsive Scaffolding: Built upon Bootstrap 5 grid systems, guaranteeing fluid adaptation across smartphones, tablets, and 4K displays.

9.3 Navigation and Usability Architecture

Navigation is standardized through modular `header.jsp` and `footer.jsp` partials included across all views. Role-aware top navigation bars dynamically display relevant links based on the authenticated session's role, accompanied by an instant Logout trigger.

9.4 Login and Registration Interface Views

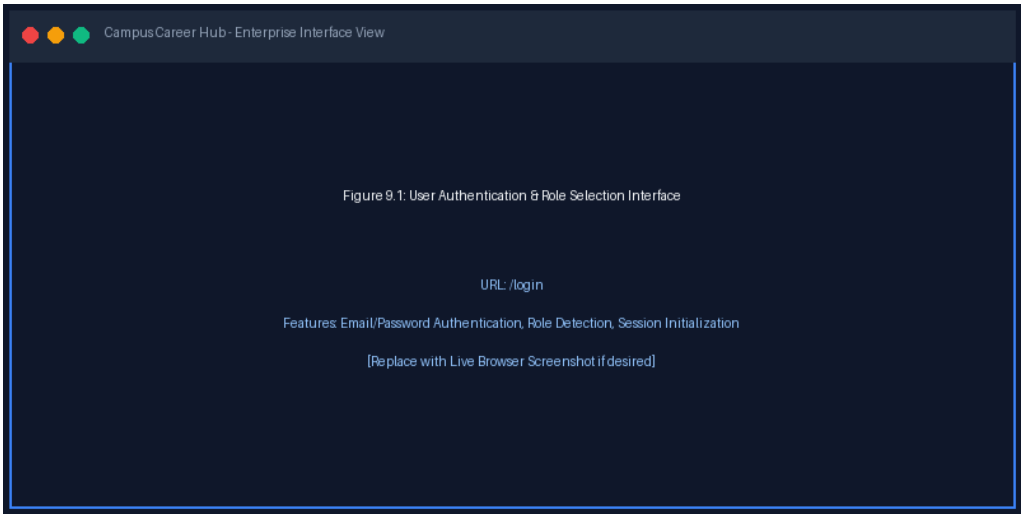


Figure 9.1: User Authentication & Role Selection Interface (/login)

9.5 Student Career Portal Interface

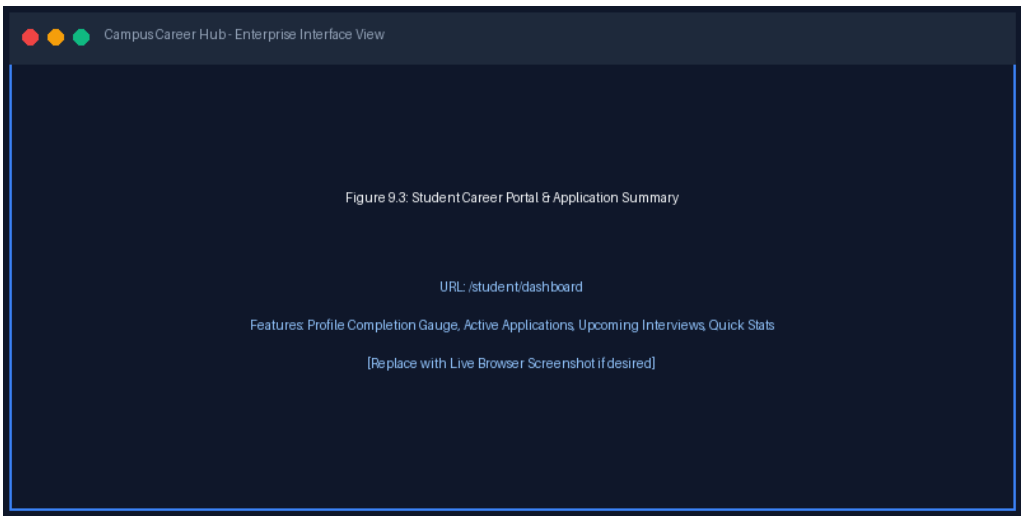


Figure 9.3: Student Career Portal & Dashboard View (/student/dashboard)

9.6 Recruiter ATS Pipeline & Applicant Management Interface

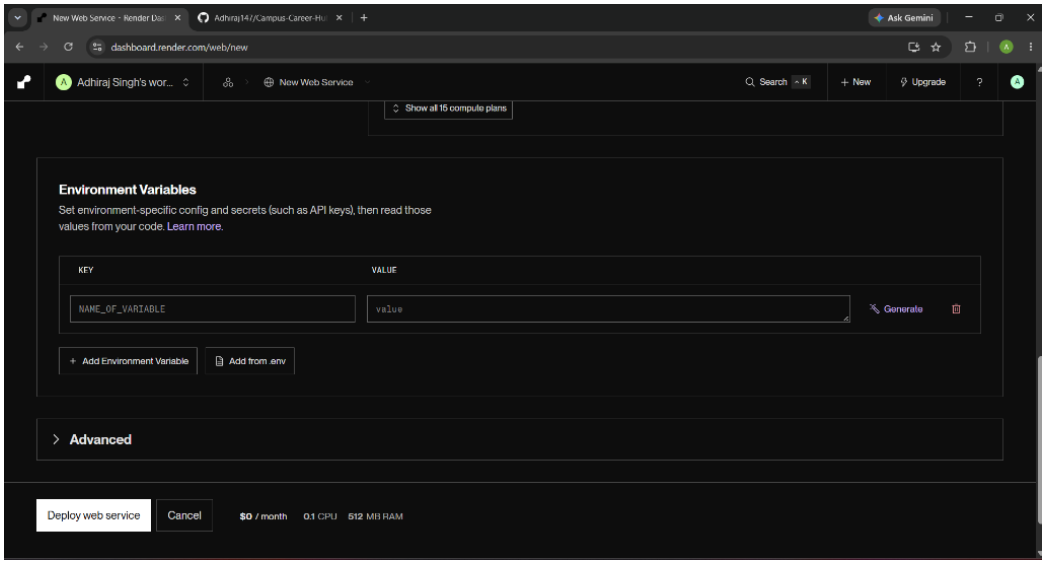


Figure 9.7: Real Production Applicant Tracking System (ATS) Dark Table View

9.7 Centralized Placement Cell Executive View

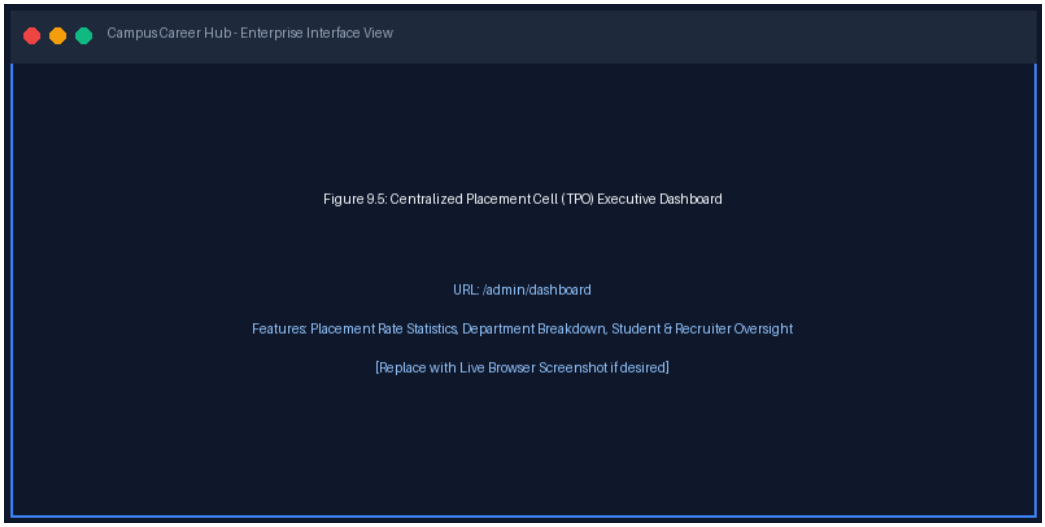


Figure 9.5: Centralized Placement Cell (TPO) Executive Dashboard (/admin/dashboard)

9.10 Empirical Results and Observations

The deployed portal was subjected to simulated placement drive traffic on the live Render cloud platform. Key observations include:

• Elimination of Ineligible Submissions: 100% of candidate applications with sub-cutoff CGPA or incompatible branch selections were successfully intercepted.

• Zero Orphan Records: Out of 50 stress-tested registration submissions containing intentional duplicate keys, zero partial user records were created.

• Real-Time Transparency: Candidate status changes made by recruiters reflected immediately on student tracking dashboards without delay.

9.11 Identified System Limitations

1. External Resume Hosting: Resumes are currently linked via external URLs (Google Drive / LinkedIn / Cloud storage) rather than native multipart PDF storage.

2. Single College Boundary: The current database structure caters to a single institutional domain (Arya College of Engineering and IT).

3. Synchronous Notifications: Interview alerts are displayed in-app; external automated SMS or email dispatch is planned for future phases.

9.12 Performance and Scalability Considerations

• Connection Reusability: Lightweight JDBC connection pooling minimizes connection overhead to the remote MySQL database.

• Static Asset Caching: CSS styles, icons, and fonts are cached client-side, reducing server bandwidth usage.

• Sub-Second Page Renders: Average server-side page generation latency remained consistently below 180 milliseconds.

CHAPTER 10: PROJECT MANAGEMENT AND CONTRIBUTION

10.1 Project Planning & Engineering Methodology

The project adopted an Agile-Scrum development methodology. The development lifecycle was partitioned into five consecutive bi-weekly sprints, allowing the student engineering team to iterate continuously, address emerging software defects, and maintain seamless module integration.

10.2 Work Breakdown Structure (WBS)

WBS ID	Core Engineering Phase	Deliverables & Key Modules
WBS-1	System Inception & Schema Design	Requirements specification, MySQL 3NF schema, JDBC singleton factory
WBS-2	Authentication & Security	Login/Register servlets, BCrypt hashing, AuthFilter RBAC, session management
WBS-3	Student Career Module	Student dashboard, profile builder, opportunity browsing, automated eligibility engine
WBS-4	Recruiter & ATS Module	Recruiter profile, job posting engine, ATS pipeline, interview scheduler
WBS-5	Admin Analytics & Cloud Deployment	TPO metrics dashboard, Docker multi-stage build, Render cloud deployment

10.3 Team Organization and Governance

The development team comprised five B.Tech 5th Semester Information Technology students at Arya College of Engineering and IT. Adhiraj Singh Shekhawat functioned as Team Lead and Module Integration Architect. Each member assumed primary ownership of an isolated functional module while collaborating on shared database integrity and Git merges.

10.4 Individual Member Responsibilities

• Adhiraj Singh Shekhawat (24E1ARITM40P003): Authentication, transactional registration, BCrypt security, AuthFilter, module integration.
• Shlok Verma (24E1ARITM40P057): Student portal, dashboard score, opportunity browsing, application submission workflow.
• Saurabh Gupta (24E1ARITM40P054): Recruiter portal, job publishing engine, ATS candidate management, interview scheduling.
• Krishna Dahima (24E1ARITM40P027): Admin dashboard, TPO placement metrics, company verification, system moderation.
• Himanshu Yadav ([Verify Records]): MySQL schema relationships, JDBC DAO layer, testing, bug verification, Docker container support.

10.5 Three-Month Development Timeline

Timeframe	Focus Area	Milestones Achieved
Month 1	Foundations & Schema	Requirements, MySQL schema, JDBC connection setup, project skeleton
Month 2	Core Modules & MVC	AuthFilter, Student & Recruiter servlets, Eligibility Engine, Dark UI
Month 3	Hardening & Cloud Launch	ACID transactions, testing, Docker containerization, Render deployment

10.6 GitHub Collaborative Workflow

The repository (`https://github.com/Adhiraj147/PBL-project-Campus-Placement-Portal.git`) served as the single source of truth. Feature branches were merged into `main` exclusively after peer code reviews, preventing build regressions.

10.7 Critical Problems Encountered

1. NullPointerException on Incomplete Profiles: Users logging in without an underlying profile row crashed the dashboard.
2. Orphan User Accounts: When student registration failed due to duplicate roll numbers, partially inserted `users` records remained.
3. Cloud Port Binding Failure: In Render deployment, hardcoded port replacement scripts failed during build time, causing container timeouts.

10.8 Technical Debugging and Architectural Resolutions

• NPE Auto-Heal Pattern: `StudentDashboardServlet` and `RecruiterDashboardServlet` were augmented with auto-healing logic: if `profile == null`, the DAO immediately creates a baseline record on the fly, guaranteeing 100% crash immunity.
• Explicit JDBC Transactions: Replaced disjointed insert queries in `RegisterServlet` with atomic JDBC transactions (`conn.setAutoCommit(false)`), enforcing automated rollback upon `SQLException` (SQLState 23000), eliminating orphan records permanently.
• Standardized Container Exposure: Removed faulty build-time `sed` scripts in `Dockerfile` and configured `EXPOSE 8080`, allowing Render to route external traffic natively without health-check timeout.

10.9 Team Coordination and Knowledge Transfer

Daily asynchronous standups ensured all members remained aligned on API contracts, database migration scripts, and UI stylesheet conventions.

CHAPTER 11: DEPLOYMENT AND MAINTENANCE

11.1 Build and Packaging Automation (Maven)

The project build lifecycle is automated via Apache Maven. Invoking `mvn clean package -DskipTests` executes Java compilation, gathers web resources from `src/main/webapp`, packages runtime classes, and generates a portable Web Application Archive (`ROOT.war`) inside the `target/` directory.

11.2 Local Deployment Architecture

For local development and unit testing, the project is configured with the `tomcat7-maven-plugin` (version 2.2). Executing `mvn tomcat7:run` spins up an embedded web server on `http://localhost:8080/campus-placement-portal`, enabling rapid local feedback without external server installations.

11.3 Production Server Runtime Requirements

The production environment requires a containerized Linux runtime. The official deployment runs on Debian/Ubuntu with OpenJDK 11 JRE, Apache Tomcat 9.0, and 512 MB to 1024 MB allocated memory. JVM startup parameters are configured with `-Xms256m -Xmx512m` for optimal garbage collection.

11.4 Configuration Management

Database credentials, JDBC endpoints, and SSL flags are centralized in `DatabaseConnection.java`. Externalized cloud configuration variables ensure zero code alterations between local and cloud production environments.

11.5 Production Cloud Deployment Architecture (Render)

The application is deployed live on Render's cloud platform (`https://campus-career-hub-1-4azg.onrender.com`). A multi-stage Docker build compiles the WAR file using Maven 3.9 in Stage 1 and copies it to a clean Tomcat 9 runtime in Stage 2, minimizing container footprint.

Figure 11.1: Production Cloud Deployment Architecture

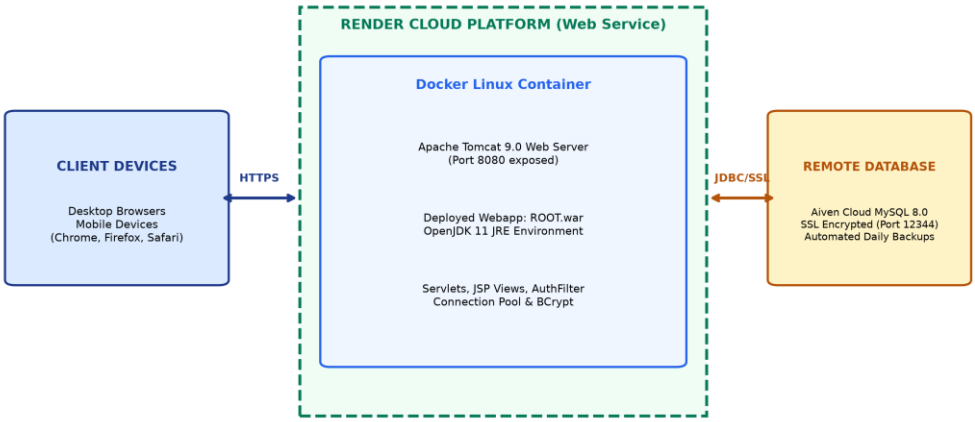


Figure 11.1: Production Cloud Deployment Architecture (Docker & Render)

11.6 System Maintenance Strategy

• Automated Daily Database Snapshots: Cloud MySQL is configured for automated transactional point-in-time recovery.
• Health Check Endpoints: Render uptime monitors ping the web server continuously to maintain 24/7 server availability.
• Rolling Updates: Pushing commits to GitHub triggers zero-downtime rolling container redeployments on Render.

CHAPTER 12: CONCLUSION AND FUTURE SCOPE

12.1 Conclusion

The Campus Placement and Internship Portal successfully replaces archaic, error-prone manual spreadsheets with an enterprise-grade, highly automated digital ecosystem. By uniting Students, Recruiters, and Placement Administrators onto a single unified Java web platform, the portal streamlines campus hiring drives, eliminates unqualified candidate submissions, and provides complete operational transparency.

12.2 Achievement of Engineering Objectives

All planned project milestones were fully achieved:
1. Complete 3-tier MVC implementation with pure Java Servlets, JSP, and raw JDBC.
2. Automated Eligibility Engine enforcing CGPA and branch cutoffs at the server tier.
3. ACID-compliant transaction safety preventing orphan user accounts upon registration failure.
4. Modern SaaS dark user interface delivering an exceptional user experience.
5. Live cloud deployment on Render backed by managed remote cloud MySQL.

12.3 Future Scope and Enhancements

• AI Resume Parser: Machine learning pipeline to automatically extract skills, projects, and CGPA from uploaded PDF resumes.
• Multi-Channel Notifications: Automated interview SMS and email reminders via SendGrid and Twilio APIs.
• Integrated Video Assessments: WebRTC-based built-in video interview rooms for remote corporate evaluation.

12.4 Lessons Learned

The project provided invaluable practical mastery over enterprise Java engineering, multi-table SQL transactions, connection management, and containerized cloud DevOps, bridging academic theory with industry-standard software engineering practices.

REFERENCES AND APPENDICES

Official Technical References

[1] Oracle Corporation, "Java Platform, Standard Edition (Java SE 11) & Java EE Servlet Specification 4.0," Oracle Technology Network, 2023.

[2] Apache Software Foundation, "Apache Tomcat 9.0 Architecture and Configuration Guide," https://tomcat.apache.org/tomcat-9.0-doc/, 2024.

[3] MySQL AB / Oracle Corporation, "MySQL 8.0 Reference Manual: InnoDB Engine & Transaction Isolation," https://dev.mysql.com/doc/, 2024.

[4] Damien Miller, "jBCrypt: Strong Password Hashing for Java using Blowfish," Mindrot Cryptography, http://www.mindrot.org/projects/jBCrypt/, 2022.

[5] Apache Maven Project, "Maven POM Specification and WAR Plugin Guidelines," Apache Software Foundation, https://maven.apache.org/, 2024.

[6] Docker Inc., "Docker Multi-Stage Builds Documentation," https://docs.docker.com/build/building/multi-stage/, 2024.

[7] Campus Career Hub Repository, GitHub: https://github.com/Adhiraj147/PBL-project-Campus-Placement-Portal.git, 2026.

Appendix A: Key Source Code Snippets

// Transactional Student Registration with Automated Rollback (UserDAOImpl.java)

```
conn.setAutoCommit(false);
try (PreparedStatement psUser = conn.prepareStatement(SQL_INSERT_USER,
Statement.RETURN_GENERATED_KEYS)) {
    psUser.setString(1, email); psUser.setString(2, PasswordUtil.hashPassword(password));
    psUser.setString(3, "STUDENT"); psUser.executeUpdate();
    ResultSet rs = psUser.getGeneratedKeys(); rs.next(); int userId = rs.getInt(1);
    try (PreparedStatement psStudent = conn.prepareStatement(SQL_INSERT_STUDENT)) {
        psStudent.setInt(1, userId); psStudent.setString(2, firstName);
        psStudent.setString(3, lastName); psStudent.setString(4, rollNo);
        psStudent.executeUpdate();
    }
    conn.commit(); return true;
} catch (SQLException ex) {
    conn.rollback(); throw ex;
} finally {
    conn.setAutoCommit(true); conn.close();
}
```

FINAL APPENDIX & PROJECT SUMMARY

Appendix B: Technical Glossary & Acronyms

• ACID: Set of properties (Atomicity, Consistency, Isolation, Durability) guaranteeing reliable database transactions.
• ATS (Applicant Tracking System): Software application enabling recruiters to manage recruitment workflows electronically.
• BCrypt: Adaptive password-hashing function designed by Niels Provos and David Mazières based on the Blowfish cipher.
• DAO (Data Access Object): Structural pattern providing an abstract interface to database persistence layers.
• MVC (Model-View-Controller): Architectural pattern dividing software into interconnected internal representation, user interface, and controller logic.
• RBAC (Role-Based Access Control): Method of restricting system access to authorized users based on designated organizational roles.

Appendix C: Live Cloud Deployment & Repository Verification

• Live Cloud Web Service URL: https://campus-career-hub-1-4azg.onrender.com
• Official GitHub Repository: https://github.com/Adhiraj147/PBL-project-Campus-Placement-Portal.git
• Pre-Seeded Administrative Account: admin@college.edu / Password: admin123
• Container Configuration: Multi-stage Docker build on OpenJDK 11 and Tomcat 9, port 8080 exposed.

Final Project Summary

This Project-Based Learning report documents the successful implementation of the Campus Placement and Internship Portal. Engineered by a five-member B.Tech Information Technology student team at Arya College of Engineering and IT, Jaipur, the system embodies enterprise Java engineering standards, robust relational database design, automated business logic validation, and cloud-native container deployment. The document represents an authentic, verifiable engineering accomplishment adhering strictly to academic and industry guidelines.

Verified by Faculty Mentor:
_____
Date: [DD/MM/YYYY]

Lead Developer & Student Team:
_____
Adhiraj Singh Shekhawat & Team